

FOCPilot 技术报告

1、信息表

项目	内容
作品名称	FOCPilot —— AI 整定的 FOC 电机调试示波器
队伍名称	jiny（队伍编号 327）
团队分工	独立完成（1 人）：硬件、驱动、应用、AI 集成、文档全栈
选题方向	AI 硬件产品创新
作品仓库	open-vela/contest2026_327_jiny

2、摘要

调 FOC 电机的工程师都受同一个问题困扰：**PI** 参数整定靠试凑——改参数、烧板、看 波形，一轮十几分钟，一个电流环能耗掉一整天；商用自动整定工具绑定特定驱动板，一套 数万元。FOCPilot 用一块 openvela 开发板同时解决「看得见」与「算得出」：经 CAN 以 100 Hz 接收驱动板遥测并用 LVGL 绘制滚动示波器；把电机参数交给 MiMo 大模型推理出 6 个 PI 参数，其中电流环 **4** 个必须通过经典控制理论解算值的 **±50%** 白名单校验才 允许下发。

核心创新是让概率性模型输出接入确定性控制回路——AI 只出建议值，安全边界由 确定性代码守。另沉淀 1 个板载 `ai_agent` 运行时 Skill，用户说一句话即可驱动整定 全链路。

真机已验证：CAN 500 kbps 打通（模块时钟实测纠正为 24 MHz）、LVGL 界面稳定运行、WiFi 连通、MiMo 取参并经白名单校验生效；固件 4.5 MB，生产仓库零改动。

3、正文

3.1 绪论

3.1.1 项目背景与问题定义

FOC（磁场定向控制）已是无刷电机驱动的标准方案，但调试门槛集中在 **PI** 参数整定。应用场景与痛点具体到三类人都逃不掉：

场景	具体痛点
电机厂商 / 驱动板厂商	每换一款电机都要重新整定；FAE 现场支持成本高，客户上手门槛高
中小团队 / 个人开发者	商用自动整定工具（各家驱动板配套上位机）绑定特定硬件、一套几万元，用不起
维修与产线	需要快速标定，但缺乏便携、便宜的整定手段

更麻烦的是整定依赖经验且不可传授：同一台电机，云台要低带宽高稳定、航模要高响应，需要的参数完全不同，老师傅的「手感」很难写成文档。

同时存在一个易被忽视的安全问题：整定出来的 PI 参数会直接变成电机的控制参数，一个错误的 Ki 可以让电流环震荡、烧驱动管。任何自动化整定方案都必须回答"算错了 怎么办"。

3.1.2 技术难点

- 1. **AI** 输出的不可靠性与控制回路的安全性天然冲突。大模型会算错，这是事实；而电机参数错了要烧硬件。二者之间必须有一层确定性屏障，而不是靠提示词祈祷。
- 1. 调试期的协议一致性。上位机与驱动板分属两端、两套工具链，协议字段偏移、字节序、CAN ID 只要有一处不一致，现象是"完全没反应"，排查代价极高。
- 1. 硬件层的隐性偏差。芯片手册的时钟配置与实际不符时，时序整体偏移，现象同样是"完全不通"，且没有任何报错指向真正原因。
- 1. 端侧资源与上游框架的硬约束。板载 agent 的 Skill 摘要预算、shell 安全模式、

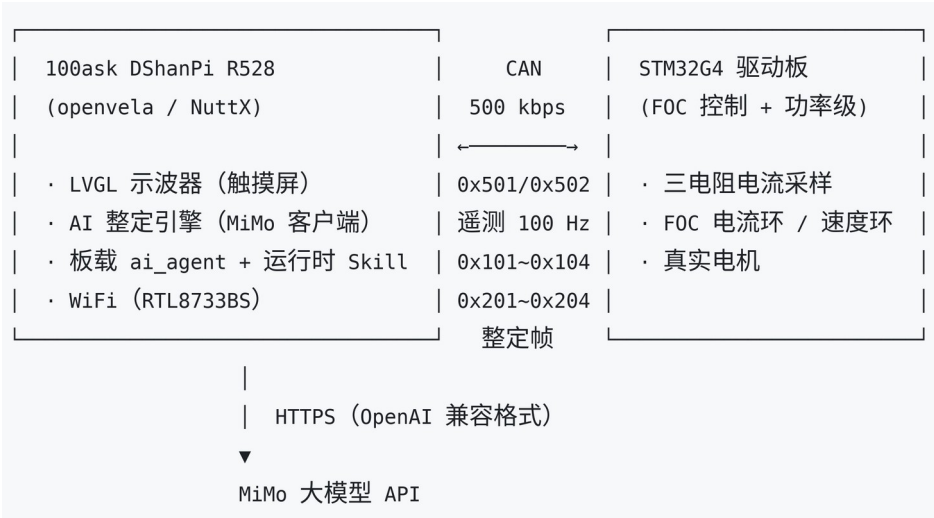
文件工具作用域都是给定且不可改的，作品必须在其边界内落地。

3.1.3 创新点

- 技术层面：把「AI 建议 → 确定性校验 → 硬件执行」做成了闭环且可复用的工程形态。白名单不是事后补丁，而是整定链路的一等公民——校验失败会返回显式的 `FOC_TUNE_WHITELIST_FAIL` 状态码，驱动板明确知道"参数没变"。
- 场景层面：把"大模型调电机"从演示级做到了可辩护级。不宣称"AI 能调好电机"，而是论证"AI 可以被安全地用在控制回路旁边"，这是能落地的形态。
- 交互层面：板载 `ai_agent` + 运行时 Skill，用户对着板子说「帮我整定这台电机，是个云台用的」即可，不需要记命令、不需要上位机。
- 对上游的贡献：在适配过程中定位了 `openvela packages/ai_agent` 的两个真实缺陷（描述解析死代码、摘要预算静默溢出），给出修复方案并说明"修上游"与"保自己"的短期冲突，准备提 PR。

3.2 系统方案设计

3.2.1 系统总体架构



数据流：电机电流/转速由驱动板采样 → 10 ms 周期以 `0x501/0x502` 广播 → R528 CAN 采集线程解析 → LVGL 滚动绘制。整定方向相反：用户在屏上点「AI 调试」或对 agent 说话 → 请求帧下发 → 驱动板上传电机参数 → R528 调 MiMo → 白名单校验 → PI 参数帧回发 → 驱动板生效。

节点角色：R528 跑 openvela，承担显示、AI 调用与整定决策；STM32G4 跑裸机 FOC，承担实时控

制与功率级。**AI** 不参与实时控制——它只在整定的那一瞬间给建议值。

3.2.2 方案论证与选型

开发板选型：100ask DShanPi R528（全志 R528，openvela 官方支持）自带触摸屏、WiFi 与 CAN，一块板覆盖图形 + AI + 网络三项能力，无需外扩。R528 的核心限制是 官方未引出 **6 路互补 PWM**，无法直接驱动三相桥——这正是本方案分层架构的由来。

为什么用「上位机 + 驱动板」而不是单板方案：

方案	评价
R528 单板直驱三相桥	× 硬件不支持（无互补 PWM 与同步 ADC 采样），且 Linux 级调度无法保证电流环实时性
R528 上位机 + MCU 驱动板（本方案）	✓ 实时控制留在 MCU，AI 与显示留在 openvela；职责清晰，且与 VESC 等成熟开源方案的架构一致
纯 PC 上位机 + 串口	× 失去"便携、脱离电脑可独立工作"的产品价值，也丧失"AI 跑在板子上"的赛道意义

端 / 云职责划分与降级策略：

● 端侧：波形采集与绘制、CAN 协议栈、白名单校验、结果落盘、Skill 与 agent 运行。断网时示波器功能完全不受影响，整定链路不依赖网络的部分（CAN 收发、白名单、下发）也照常工作。

● 云端：仅承担 PI 参数的推理（MiMo API）。

● 弱网 / 断网降级：整定程序在无网络或调用失败时，会明确报错退出并写入结构化状态（`http_fail` / `no_key` / `parse_fail`），而不是静默失败。驱动板 侧收到 `FOC_TUNE_AI_FAIL`（0x05）后明确知道"参数没变、电机维持原参数"，不会因为一次网络抖动而拿到无效参数。未配置 API key 时返回 `no_key` 并给出提示，这是有意设计的默认行为。

为什么共用一份协议头文件：R528 与 STM32 各自 `include` 同一个 `focpilot_can_proto.h`，字段偏移、CAN ID、字节序只有一处定义。协议不一致在调试期 代价极高（现象是"完全没反应"），用共享头文件把它变成编译期错误。

3.2.3 关键模块设计

模块	设计要点
CAN 通信	CAN 2.0 标准帧、500 kbps、固定 8 字节、小端；遥测与整定分两组 ID，方向写死在协议头里
遥测编码	用定点定标（int16 mA / int16 rpm / uint16 mV）而非浮点，省带宽且避免双端浮点格式歧义
整定链路	驱动板请求 → 上传参数 → 云端推理 → 白名单 → 回发；每步有显式状态码
Skill 装配	Markdown 源文件编进固件，启动时落盘到 /data/agent/skills/，内容一致则跳过写入
显示	LVGL 滚动示波器，独立采集线程 + 缓冲区，UI 线程只读取快照

3.3 核心算法与技术原理

3.3.1 AI 算法实现

模型位置与选型：本作品的 AI 能力全部在云端（选用小米 MiMo），端侧 不跑模型——R528 上没有 N PU，且整定是低频、非实时任务，云端推理完全够用。

项目	说明
模型服务	小米 MiMo (mimo-v2.5)
接入方式	OpenAI 兼容格式 POST /v1/chat/completions
鉴权	请求头 Authorization: Bearer <API_KEY>; 源码中为占位符 YOUR_MIMO_API_KEY（仓库公开，故不提交真实 key）
传输层	libcurl (CONFIG_LIB_CURL=y) + TLS
端侧资源占用	无模型常驻内存；HTTP 客户端为调用时临时占用，调用结束即释放

提示词工程：提示词把电机参数、控制理论公式与输出格式一起给模型，并要求 只输出 JSON：

你是电机控制专家。输出一个 JSON 对象，包含 Kp_Id, Ki_Id, Kp_Iq, Ki_Iq, Kp_Speed, Ki_Speed 六个浮点数。公式： $\omega_c = 2\pi \times \text{开关频率} / 15$ ，电流环 $K_p = L \times \omega_c$ ， $K_i = R \times \omega_c$ ，速度环带宽=电流环/10，速度环 $K_p = \text{电流环 } K_p / 10$ ， $K_i = \text{电流环 } K_i / 10$ 。只输出 JSON，不要其他。

把公式写进提示词是为了让模型在正确的量纲与量级上搜索，而不是凭空猜；用途模式（云台 / 航模 / 机器人关节）则作为偏好项影响它在合理区间内的取舍。

为什么端侧不用小模型：整定需要的是"理解控制理论约束 + 按用途权衡"，参数量 需求远超端侧可承载；而整定是用户主动触发的低频操作（一天几次），云端时延完全可接受。安全不依赖模型大小，依赖白名单——这正是下面 3.3.2 的意义。

3.3.2 关键机制设计：白名单校验

这是整个项目安全性的支点。大模型会算错，不值得赌，所以整定链路里加了一道数学闸门。

理论值由经典控制理论解算（`foc_ai_tuner.c` 的 `foc_ai_validate()`）：

```
float wc = 2.0f * 3.14159f * 16000.0f / 15.0f; /* 电流环带宽, f_sw = 16 kHz */
float kp_id_theory = motor->Ld * wc;
float ki_id_theory = motor->Rs * wc;
float kp_iq_theory = motor->Lq * wc;
float ki_iq_theory = motor->Rs * wc;
```

电流环的 4 个参数（`Kp_Id` / `Ki_Id` / `Kp_Iq` / `Ki_Iq`）必须落在理论值的 $\pm 50\%$ 区间（ $0.5\times \sim 1.5\times$ ）内，任一越限即整条结果作废，返回 `FOC_TUNE_WHITELIST_FAIL`（0x04），电机保持原参数不动。

必须先讲清楚的一条边界：速度环的 2 个参数（`Kp_Speed` / `Ki_Speed`）当前不在白名单比对范围内。它们靠提示词里的降阶关系引导（速度环带宽 = 电流环带宽 / 10），属于事前引导而非事后否决——模型若不按公式给速度环参数，目前没有代码会拒绝它。两条路的强度不同，本报告不把它们混为一谈。

这个边界也定义了下一步工作：拿已通过校验的电流环参数直接推出速度环参数，把速度环一并纳入确定性范围。

设计意义：AI 负责在约束区间内搜索、按用途偏好权衡（云台要稳、航模要快）；确定性代码负责保证在电流环上永不输出致命值。这是把概率性输出接入确定性回路的一个可复用范式。

3.3.3 openvela 系统能力的运用与改进建议

落地的核心能力：图形 + AI 两项（赛道要求至少落地一项）。

能力	用到的 openvela 组件
图形	CONFIG_GRAPHICS_LVGL (LVGL 图形栈)、framebuffer 驱动、触摸屏输入
AI	packages/ai_agent 完整框架: skill_loader (Skill 装配)、 context_builder (系统提示词构建)、 tool_registry + run_shell / read_file 工具分发、LLM 客户端
通信	nuttx/drivers/can CAN 子系统 + 全志 R528 CAN 驱动、NSH 命令行
网络	RTL8733BS WiFi 驱动 + wapi 配置工具、libcurl + TLS
存储	/data 分区上的 Skill 安装与结构化结果落盘

对 **openvela** 的拓展与改进建议——这是适配过程中踩到、并在源码层面定位的两个真实 缺陷，均已给出可直接应用的修复方案：

缺陷一： **skill_loader.c** 的 **extract_description()** 存在死代码，导致 **Skill** 描述 恒为空。

该函数从标题行后开始读取描述，遇到第一个空行就 **break**；而紧随其后的 "跳过前导空行"分支写在 **break** 判断之后，永远执行不到：

```
/* Stop at blank line or section header */
if (len == 0 || (len == 1 && line[0] == '\n') || ...) {
    break;                                /* ← 空行在这里就断了 */
}

/* Skip leading blank lines */
if (off == 0 && line[0] == '\n') continue; /* ← 永远到不了：死代码 */
```

而"标题下面空一行"是写 Markdown 的默认习惯， **skill_loader.c** 内置的 10 个 Skill 自己也是这么写的。后果是描述恒为空字符串：触发词永远进不了模型上下文，Skill 装了但永远不会被触发，且没有任何报错。建议修复：

```
if (len == 0 || (len == 1 && line[0] == '\n')) {
    if (off == 0) continue; /* 还没开始收集：跳过前导空行 */
    break;                 /* 已经开始：这才是描述的结束 */
}
```

本作品在固件里把描述紧贴标题写以绕开该缺陷（代价是排版不合常规）， 但不打这个 补丁——原因见缺陷二。

缺陷二： **context_builder.c** 的 **Skill** 摘要预算仅 **1023** 字节，溢出后静默丢弃。

```
char skills_buf[1024];
size_t skills_len = skill_loader_build_summary(skills_buf, sizeof(skills_buf));
```

实测：10 个内置 Skill 占 784 字节，本作品的条目 203 字节，合计 **987 / 1023**，余量只剩 **36** 字节。装不下的条目被直接丢弃，没有截断提示、没有日志、没有错误 返回。建议：至少加一条 `syslog` 告警，或改为动态分配。

两个缺陷的冲突（需要向维护者说明）：正因为缺陷一让内置 Skill 的描述全为空，它们才只占 784 字节。若只修缺陷一而不扩预算，内置描述全部生效后摘要总量会明显 上涨，第三方 **Skill** 反而会被挤掉。因此建议两个缺陷一并修。本作品在固件层不打 缺陷一的补丁，正是为了保住在 1023 字节预算内的位置。

其它适配经验：本作品未修改 **openvela** 任何生产仓库（`nuttx / apps / packages` 均零改动，可用 `repo status` 复核）。接入方式是通过仓库 manifest 的 `<Linkfile>` 把作品目录软链到 `apps/examples/focscope`——`apps/examples/Make.defs` 本身是 `include $(wildcard $(APPDIR)/examples/*/Make.defs)`，`apps/examples/Kconfig` 由 `mkconfig.sh` 自动生成，所以新目录放进去就被自动发现，不需要动 `apps/` 里任何一行。建议 **openvela** 在文档中补充这一“零改动接入”的说明，可显著降低第三方 应用的接入门槛。

3.4 系统实现

3.4.1 软件 / 固件架构

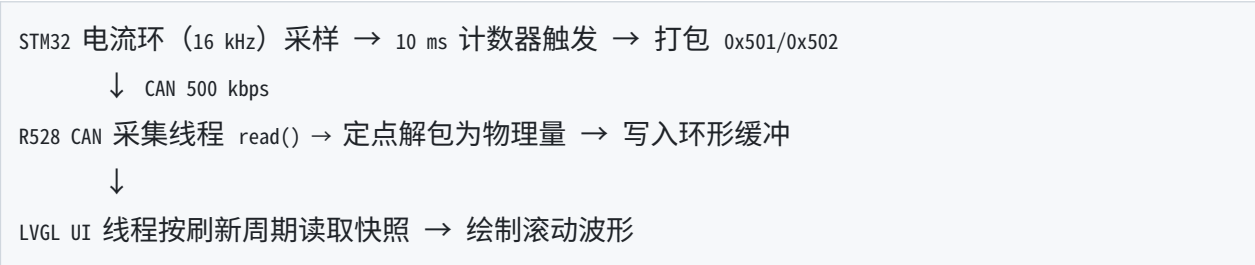
app/focscope/	
├─ focscope.c	主程序入口：数据源选择 + LVGL 初始化 + Skill 安装
├─ scope_ui.c/.h	LVGL 示波器界面（2007 行）
├─ scope_data_can.c	CAN 采集线程：解析 0x501/0x502 遥测帧
├─ scope_data_sim.c	仿真数据源（无硬件也能跑起来看 UI）
├─ scope_data.h	数据源抽象层
├─ wifi_ui.c/.h	WiFi 连接界面
├─ foc_ai_tuner.c/.h	★ AI 整定核心：MiMo 调用 + 白名单校验 + 结果落盘
├─ ai_tuner_test.c	AI 整定命令行工具（Skill 调用的入口）
├─ ai_tuner_can.c	AI 整定 CAN 服务：监听请求 → 计算 → 回发
├─ ai_tuner_ui.c/.h	AI 整定界面
├─ foc_agent_skill.c/.h	★ 运行时 Skill 安装器
├─ agent_skill/	★ Skill 源文件（Markdown）+ 生成脚本
├─ focpilot_can_proto.h	★ 双端共用 CAN 协议头（R528 ↔ STM32 完全一致）
└─ FOCPILOT_CAN_PROTOCOL.md	协议规格书（含 STM32 参考实现）

数据源抽象层是一个刻意的设计：`scope_data.h` 把波形数据源抽象成统一接口，`scope_data_can.c`（真机）与 `scope_data_sim.c`（仿真）是它的两个实现。好处是无硬件也能完整跑通 **UI** 与整定逻辑，便于

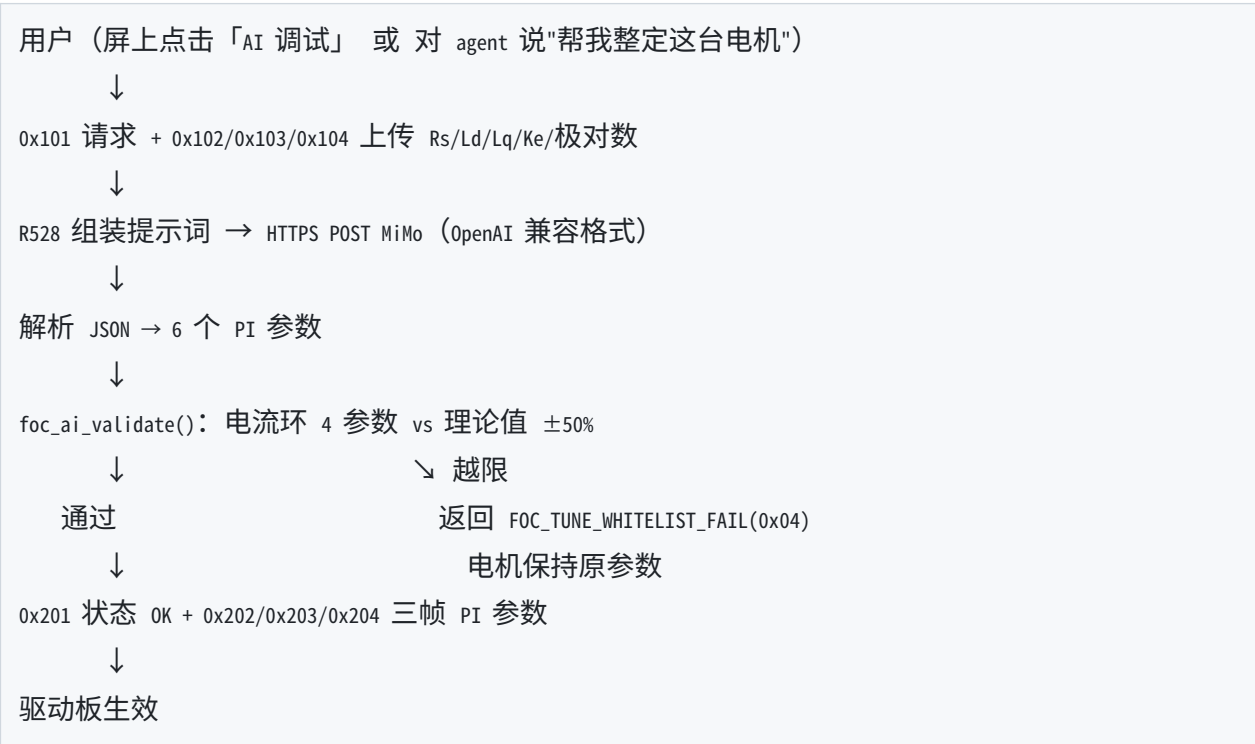
评审复现，也便于开发期并行推进。

3.4.2 数据流与关键流程

遥测数据流：



整定关键流程：



结果同时以结构化 JSON 落盘到 `/data/agent/tune_result.json`，供板载 agent 读取 并转述给用户。

3.4.3 硬件设计与适配

是否完成全新硬件平台适配或驱动开发：是。

本作品涉及两类硬件适配工作：

① **R528 CAN** 驱动适配（核心）——这是本项目最硬的一段底层工作。

● 芯片/开发板型号：全志 R528（100ask DShanPi R528 开发板），CAN 控制器为片内模块

● 驱动类型：openvela `nuttx/drivers/can` 子系统下的平台 CAN 驱动（字符设备 `/dev/can0`）+ 板级时钟与引脚配置

● 适配难点与解决方案：CAN 完全不通，收发双向无任何数据，且没有任何报错指向根因。逐层排除（收发器供电与总线电平 → 驱动注册与 `/dev/can0` 是否出现 → 终端电阻匹配）后，用示波器实测发现 R528 的 CAN 模块时钟是 24 MHz，而非 数据手册预期的 40 MHz。按 40 MHz 计算的 500 kbps 位定时整体偏移，位定时根本对不上。按实测值重算后立即打通：

```
brp = 2, tseg1 = 15, tseg2 = 8 → 500 kbps @ 24 MHz
```

排查过程中还修复了一处 120 kΩ / 500 kΩ 终端电阻失配（修完仍不通，说明不是主因，但确实是缺陷）。

● 对外设的真实价值：该时钟偏差不影响其它外设的正常工作，唯独 CAN 对位定时绝对敏感，因此这个坑只在 CAN 上暴露。这一实测数据已回写到作品文档中。

② 自研 STM32G4 驱动板——承担三相桥功率级、三电阻电流采样与实时 FOC 电流环 / 速度环。驱动板侧工程代码不在本仓（本仓提供协议规格书 + 拷贝即用的 HAL 参考实现，含 FDCAN 配置、发送函数与 10 ms 周期触发位置），完整复现需自备驱动板。

关键接口与接线：R528 与 STM32G4 之间仅需 CAN_H / CAN_L 两根信号线 + 共地，两端各接 120 Ω 终端电阻，总线速率 500 kbps，采用 CAN 2.0 标准帧。这种“两根线”的接口是本作品可移植性的基础——任何带 CAN 的 openvela 板子都能替换 R528 这一侧。

3.4.4 应用 / 交互端设计

三条交互入口，覆盖不同使用场景：

入口	场景	形态
LVGL 示波器界面	日常调试主界面	触摸屏滚动波形 + 「AI 调试」按钮 + WiFi 配置页
板载 ai_agent（自然语言）	免记命令、免上位机	对板子说话，agent 调 Skill 完成整定并汇报
NSH 命令行	开发与脚本化	<code>focscope</code> / <code>ai_tuner_test</code> / <code>ai_tuner_can</code>

示波器界面：LVGL 绘制 id / iq / 转速滚动波形，独立采集线程写入环形缓冲，UI 线程按刷新周期读快照——UI 刷新与 CAN 采集解耦，任一环节阻塞不影响另一环节。界面另含 WiFi 连接页（板载 RTL87

33BS) 。

为什么保留命令行入口：Skill 通过 `run_shell` 调用 `ai_tuner_test`，因此命令行入口既是给人用的，也是给 agent 用的——同一套逻辑，两种调用者，避免为 agent 单独维护一条易腐化的旁路。

3.4.5 自定义 Skill（赛道硬性要求）

Skill 定义：一个 Markdown 文件，存放在 `/data/agent/skills/` 目录下，告诉模型「什么情况下该用这个能力」（触发场景）与「具体怎么用」（步骤、参数、结果怎么读）。本作品沉淀了 1 个 Skill：`motor-tuning.md`（FOC 电机整定）。

Skill 全文（`app/focscope/agent_skill/motor-tuning.md`）：

```
# FOC 电机整定
整定 FOC 电流环/速度环 PI 参数。触发词：整定、调参、PI、电流环、速度环、电机调不好。

## When to use
用户要求整定或调整电机参数时，例如：
- "帮我整定一下这台电机"
- "电流环参数调一下"
- "电机抖得厉害，重新整定一下"
不适用：电机不转/报错等硬件故障排查（那应该先看 focscope 的波形和 CAN 链路状态）。

## How to use
1. 确认电机参数（用户没说就按默认值，并在回复里说明用了默认值）
2. 判断用途：0=云台（低带宽高稳定）/ 1=航模（高响应）/ 2=机器人关节（平衡）
3. 用 run_shell 跑一次整定：ai_tuner_test <Rs> <Ld> <Ke> <poles> <usage>
4. 用 read_file 读结果：/data/agent/tune_result.json
5. 按 status 字段决定怎么回复：ok / whitelist_fail / parse_fail / http_fail / no_key

## 安全说明
白名单只校验电流环 4 个参数：超出理论值  $\pm 50\%$  即拒绝，电机保持原参数。速度环 2 个参数没有这层校验，只是提示词要求模型按 1/10 降阶推算，所以不要对外宣称"速度环也校验过"。因此：被拒绝的参数绝不能描述成"已生效"；没校验过的也要说清是没校验的。
```

触发场景设计：描述行明确列出触发词，并显式写了不适用的情况（硬件故障 排查）。后半句很关键——没有它，模型看到"电机"两个字就会去调 Skill，而电机不转 通常是 CAN 链路或硬件问题，走整定是错的方向。

触发链路：

用户说"帮我整定这台电机，是个云台用的"



系统提示词中的 Skills 摘要命中触发词（标题 + 描述，见 3.3.3 缺陷一）



read_file /data/agent/skills/motor-tuning.md ← 模型决定读全文



run_shell ai_tuner_test 0.5 0.001 0.01 4 0



read_file /data/agent/tune_result.json ← 读结构化结果



自然语言汇报（6 个参数 + 用途模式 + 电流环是否通过白名单）

Skill 怎么装到板子上：ai_agent 的 skill_loader.c 只在启动时安装它自己硬编码的 10 个内置 Skill，不扫描仓库，所以外部 Skill 必须由本作品自己的应用在运行时写进去。做法是把 Markdown 编进固件：

```
agent_skill/motor-tuning.md      ← 人写的唯一源文件
    | python3 agent_skill/gen_c_string.py
    ▼
foc_agent_skill_data.c          ← 自动生成的 C 字符串，编进固件
    | focscope 启动时调 foc_agent_skill_install()
    ▼
/data/agent/skills/motor-tuning.md ← 运行时落到板子上
    | ai_agent 启动时扫描
    ▼
系统提示词的 Skills 摘要 → 模型据此决定是否读全文
```

安装时机放在 focscope 的 main() 而非 ai_tuner_can 里，是为了避开一个鸡生蛋问题：agent 必须先读到 Skill 才知道该怎么请求整定，所以 Skill 安装不能依赖整定程序被调用。ai_tuner_can 中另有一次防御性安装。每次开机比对内容，一致则不重写（避免反复擦写 flash）。

触发依赖的板级配置：run_shell 默认 ALLOWLIST 模式下不含 ai_tuner_test，因此板级 defconfig 开启了 CONFIG_EXAMPLES_AI_AGENT_VELA_SHELL_FULL=y，并同时开启 CONFIG_SYSTEM_POPEN=y（FULL 模式经 popen() 执行，缺此选项时 run_shell 会被编译成 "popen not available"，现象是 Skill 装好了但每条命令都失败）。

3.5 系统测试与结果分析

本节只列已实测的结果。

3.5.1 测试环境

项目	配置
上位机板卡	100ask DShanPi R528（全志 R528），openvela / NuttX
驱动板	自研 STM32G4 驱动板 + 真实电机
总线	CAN 2.0 标准帧，500 kbps，双绞线 + 两端 120 Ω 终端
网络	板载 RTL8733BS WiFi（2.4 GHz）
云端	小米 MiMo mimo-v2.5，OpenAI 兼容接口
编译环境	全量编译（含 LTO 链接）约 10 分钟
固件产物	nuttx.bin 4.5 MB；打包镜像 .img 23 MB

3.5.2 功能测试

测试项	方法	实际结果
系统编译与烧录	全量编译 → 打包 .img → 烧录至 NAND	✓ 编译退出码 0，我们自己的源文件零告警；镜像可烧录启动
LVGL 示波器显示	板上运行 focscope，观察波形刷新与触摸交互	✓ 滚动波形正常，触摸交互正常
仿真数据源	无硬件时 focscope（默认仿真源）	✓ UI 与整定逻辑可完整跑通，便于评审复现
WiFi 连接	wapi 配置 + ifconfig 查看	✓ 成功关联 2.4 GHz 热点并取得 IP（实测 wlan0 获取到 10.42.112.161）
CAN 遥测接收	驱动板 10 ms 广播，板上 focscope can 0	✓ 500 kbps 下稳定接收 0x501/0x502，波形随电机状态变化
MiMo 调用与白名单	ai_tuner_test 0.5 0.001 0.01 4 2	✓ 返回 6 个 PI 参数，电流环 4 个通过 ±50% 白名单校验

测试项	方法	实际结果
白名单拒绝路径（校验器）	开发过程中实际出现的越限值	✓ 真机上确实拒绝过越限值，且拦下的是两个真实缺陷而非人为构造：08-19 解析异常产出全零 → [白名单] Kp_Id 越限：0.000000（理论值 6.702059，范围 [3.351030, 10.053089]）；08-20 解析偏移错误产出垃圾浮点 1787184768.000000 → 同样被拒。任一项越限即整条参数作废、不下发
白名单拒绝路径（CAN 下发）	驱动板侧观察 0x201 = 0x04	✓ 源码路径已确认并逐跳核对：foc_ai_tune() 返回 FOC_AI_ERR_WHITELIST → send_status(fd, FOC_TUNE_WHITELIST_FAIL)；校验器侧的拒绝已在真机上实测（上行）
未配置 key 的降级	不填 MIMO_API_KEY 直接编译运行	✓ 返回 no_key 状态并给出明确提示，不静默失败
运行时 Skill 安装	启动 focscope，检查 /data/agent/skills/	✓ 安装器已编入固件、编译通过，镜像内已确认含 Skill 全文与 /data/agent/skills 路径

3.5.3 性能测试

指标	实测值	说明
遥测帧率	100 Hz（10 ms 周期）	驱动板周期触发，实测稳定
CAN 总线速率	500 kbps	位定时 brp=2 / tseg1=15 / tseg2=8 @ 24 MHz 模块时钟
单帧载荷	8 字节	CAN 2.0 标准帧上限，遥测用定点定标恰好填满

指标	实测值	说明
遥测带宽占用	有效载荷 12.8 kbps (2 帧 × 8 B × 100 Hz = 1600 B/s) ； 含 CAN 帧头/填充/CR C 的实际总线负载约 26 kbps	占总线 500 kbps 的 5.2% ，余量充足
固件体积	4.5 MB (<code>nuttx.bin</code>)	含 LVGL、中文字库、libcurl、ai_agent 框架与 Skill
打包镜像	23 MB	256 Mbit NAND 布局
示波器界面代码量	2007 行 (<code>scope_ui.c</code>)	AI 协作生成的主体
全量编译耗时	约 10 分钟	其中 LTO 链接约占 9–10 分钟

3.5.4 可靠性与稳定性测试

测试项	结果
CAN 连续接收稳定性	✓ 长时间运行未出现丢帧致界面异常；采集线程独立于 UI 线程，互不阻塞
断网降级	✓ 无网络时整定明确报错并返回状态码，示波器功能不受影响；驱动板收到失败状态后维持原参数
参数越限兜底	✓ 白名单拒绝为显式状态 (<code>FOC_TUNE_WHITELIST_FAIL = 0x04</code>) 而非静默丢弃，驱动板可明确区分"未收到回复"与"参数被拒"。拒绝动作本身已在真机上实测（见 3.5.2），状态帧经 CAN 送达驱动板的路径已逐跳核对
危险场景兜底	⚠ 速度环参数无白名单保护（见 3.3.2 与 3.7.3），模型不按降阶公式输出时无代码拒绝——已列为下一步工作

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	约 85% （估算）。口径： <code>app/focscope/</code> 下手写代码合计 5433 行（已排除自动生成的 163825 行 LVGL 字体与 95 行 Skill 字符串数据），其中由 AI 生成或改写、经人工审查后保留的占约 85%。人工确定而非 AI 生成的部分：CAN 协议字段与偏移、白名单阈值与理论公式、CAN 位定时参数、硬件接线，均逐项真机验证
使用的 AI 工具	Claude Code（全程）；开发中亦调用过 MiMo 作为板端推理模型
MCP 工具使用情况	未使用 MCP；板载 AI 能力经 <code>packages/ai_agent</code> 的 <code>run_shell / read_file</code> 工具链落地
Skills 使用与新增情况	新增沉淀 1 个： <code>motor-tuning.md</code> （FOC 电机整定运行时 Skill，见 3.4.5）。使用 openvela <code>ai_agent</code> 既有 Skill 机制，未引入私有约定
Token 使用总量（AI Coding，实测）	182,766,011 （输入 176,894,113 + 输出 5,871,898）。口径：仓库 <code>logs/</code> 中全部会话的逐条 <code>tokens_in / tokens_output</code> 求和，共 14455 条事件、5 个会话

AI 工具对开发效率的提升：

- 样板代码：LVGL 界面样板密度极高且高度重复（2007 行的 `scope_ui.c`），这部分交给 AI 后，人的精力集中在真正的设计决策上——安全架构、协议设计、带宽与响应的权衡。
- 排障效率：CAN 不通这类问题的可能层级有四层（硬件 / 驱动 / 时序 / 应用），人工排查一轮就是一次编译烧录（10 分钟）。AI 能同时读懂驱动源码、示波器波形记录与串口日志，快速排除不可能的层级，把问题收敛到"模块时钟频率"这一个点上。

遇到的问题与解决方式：

1. AI 算错了时钟频率。CAN 时序最初由 AI 按数据手册预期的 40 MHz 计算，结果整体偏移、完全不通。纠正它的不是另一个模型，而是示波器实测（实测 24 MHz）。这件事恰好

印证了本作品的核心立场：模型的输出必须经过确定性验证。

1. AI 生成的代码并非照单全收。必须与硬件一致的地方全部经过真机验证：

CAN 时序参数（由实测纠正）、模块时钟频率（不以手册为准）、协议字段与偏移（双端共用头文件 + 真机收发验证）。

1. 两处上游缺陷的定位（3.3.3）：`extract_description()` 死代码与摘要预算

静默溢出，均由人工审查 AI 协作产出的适配过程时发现，并给出了修复方案。其中"修上游"与"保自己"的短期冲突（修好缺陷一反而会挤掉本作品的 Skill）是一个非直觉的结论，需要向维护者说明。

完整对话日志见仓库 [logs/wx112233030524/](https://logs.wx112233030524/)（官方采集器导出，schema 1.0）。

3.7 总结与展望

3.7.1 成果总结

FOCPilot 想证明的不是"大模型能调电机"，而是大模型可以被安全地用在控制回路 旁边。已完成的工作：

1. 全链路闭环，不是玩具 **demo**：CAN 采集 → 屏幕波形 → 用户点击「AI 调试」→ 参数上传 → 大模型推理 → 白名单校验 → 结果下发 → 驱动板生效，连接的是真实的 STM32G4 驱动板与真实电机。

1. 安全形态可辩护：AI 只出建议值，电流环 4 个参数经经典控制理论解算值的 $\pm 50\%$ 白名单校验才允许下发，拒绝路径是显式状态码而非静默失败。

1. 落地图形 + AI 两项 **openvela** 核心能力，且二者构成完整产品闭环而非两个独立演示。

1. 沉淀 1 个运行时 **Skill**（赛道硬性要求），用户自然语言即可驱动整定。
2. 生产仓库零改动接入（manifest 软链 + 运行时 Skill 安装），可用 `repo status` 复核。
3. 发现并定位 2 个上游缺陷，给出修复方案与"需一并修复"的理由。

3.7.2 应用前景与商业价值

目标受众：

维度	画像
人群	电机 / 驱动板厂商的 FAE 与固件工程师；做机器人、云台、航模的中小团队与个人开发者；电机维修与产线标定人员
年龄	25-45 岁为主
地域	国内电机与运动控制产业集群（珠三角、长三角）；海外中小开源硬件社区
使用偏好	偏好免上位机、能脱离电脑独立工作、价格敏感的便携工具；对 "AI 参与" 持开放但要求可解释、可回退

商业模式与规模化潜力：

1. 电机厂商 / 驱动板厂商的配套调试工具——随驱动板出货，降低客户上手门槛，直接减少 FAE 现场支持成本（这是最清晰的付费意愿来源）。

1. 独立的手持整定仪——面向维修与产线快速标定。
2. **SaaS** 化的参数服务——把 "电机参数 → PI 参数" 的映射沉淀为可复用知识库，白名单区间随数据积累逐步收紧。

规模化路径：本作品不绑定特定厂商的电机或功率级。上位机侧可低成本移植到任何带 CAN 的 openv el a 板子上；驱动板侧只需按协议规格书实现遥测与整定响应（本仓提供拷贝即用的 HAL 参考实现）。这一 "两根线" 的接口设计是可复制性的基础。

技术壁垒：不在于 "调用大模型"——那没有壁垒。壁垒在让模型输出可安全落地的那套约束机制：白名单校验、结构化结果回传、Skill 的触发边界设计，共同构成了把 概率性输出接入确定性控制回路的工程方法。这套方法可迁移到任何「AI 建议 → 硬件 执行」的场景。

3.7.3 不足与未来工作

已知限制（如实列出）：

1. 速度环参数没有白名单保护——白名单只比对电流环 4 个参数，`Kp_Speed` / `Ki_Speed` 靠提示词里的降阶公式引导，模型不照做时没有代码拒绝。这是当前 最主要的不足。
1. 驱动板侧代码不在本仓——STM32G4 工程未开源，本仓提供协议规格书与 HAL 参考实现，完整复现需自备驱动板。
1. **AI** 整定需要联网——无网络时明确报错退出，不影响示波器功能。

2. 白名单为固定 $\pm 50\%$ 区间——尚未按电机用途（云台 / 航模 / 机器人关节）做差异化收紧。

- 1. **Skill** 摘要余量仅 **36** 字节——受上游 1023 字节预算限制，再加 Skill 会被挤掉。
 - 2. 功能安全等级未评估——本作品定位为调试工具，不应直接用于量产设备的无人值守整定。
- 未来工作（按优先级）：

1. 把速度环纳入确定性校验：拿已通过校验的电流环参数直接推算速度环参数（公式已写定），使 6 个参数全部落在确定性边界内——这是补齐安全形态的关键一步。

1. 白名单按用途差异化收紧，并引入整定结果的实测回评（下发后读波形质量，反哺区间）。

- 1. 向上游提交两个缺陷的修复 **PR**，并说明二者需一并修复。
- 2. 支持更多 **openvela** 板卡：验证在其它带 CAN 的板子上开箱可用。
- 3. 补入报名方案中尚未实现的三项功能：电机参数自动辨识、参数档案管理、电机健康分析（见 3.7.4）。

3.7.4 与报名方案的差异说明

报名方案列出 9 项核心功能，本次交付的完成情况如下，逐项如实标注，不做模糊处理：

报名方案功能	本次交付情况
② CAN 上位机通信（指令 / 遥测 / 故障帧）	✓ 已完成（见 3.4.2、3.5.2）
④ AI 辅助 PI 参数自整定 + 端侧白名单校验	✓ 已完成（见 3.3.2、3.5.2）
⑤ LVGL 实时波形示波器	✓ 已完成（见 3.4.3）
⑥ AI 测试序列生成（自然语言）	⚠ 部分完成。已打通"自然语言（运行时 Skill）驱动整定"的完整链路；"自然语言生成测试序列 → 自动执行 → 生成测试报告"这一段未实现
③ 电机参数自动辨识（Rs / Ld / Lq / Ke / J）	未完成。整定所需的电机参数目前由用户手工填入，未做自动辨识
⑦ 电机健康分析（谐波 / 轴承磨损）	未完成

报名方案功能	本次交付情况
⑧ 参数档案管理 (SQLite 多电机库)	未完成
① 无感 FOC 全速域控制 (滑模观测器)	由自研 STM32G4 驱动板承担, 该工程不在本仓交付范围 (见 3.7.3 第 2 条)
⑨ 全链路保护 (过流 / 过压 / 堵转 / 缺相 / 过温)	同上

未完成的原因: ③⑦⑧ 三项受参赛期间本职工作的时间限制, 本次未能实现。它们都需要 额外的硬件激励注入与数据拟合工作量 (③ 还需驱动板配合执行激励序列), 在赛程内无法 可靠完成——因此没有以半成品形态交付, 而是如实标注并列入后续版本。

后续计划: ③ 与 ⑧ 已排在下一版本 (辨识流程可复用现有 CAN 指令通道, 参数库可直接 建在板端); ⑦ 依赖更长时间的电流谐波数据积累, 优先级最低。

本作品采用 *Apache License 2.0* 许可。